CRESCENDO®
INTERNATIONAL
COLLEGE
IPTS No. DK006(J)

Advanced Web Development

SYNCHRONOUS VS ASYNCHRONOUS WEB SERVICES

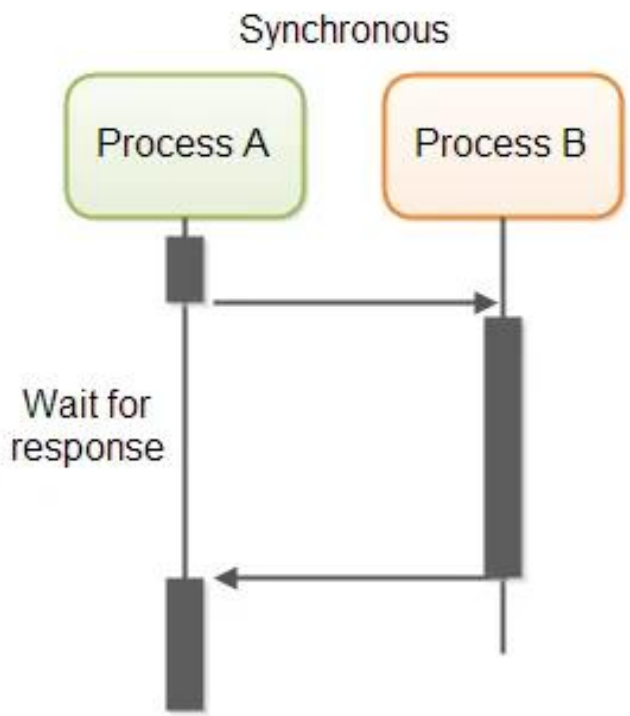
CRESCENDO®
INTERNATIONAL
COLLEGE
IPTS No. DK006(J)

What is Web Service?

- A **web service** is any piece of software that makes itself available over the internet and uses a standardized XML messaging system.
- Web service is a software service **used to communicate between two devices** on a network.
- Web service is a **software application** with a standardized way of providing interoperability between disparate applications.

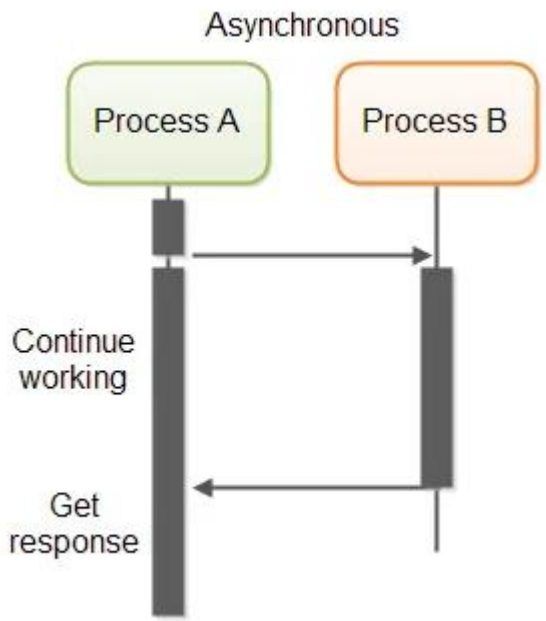
Synchronous Web Services

- This service execution provides an **immediate response to a query**.
- Client will **wait until the server sends back the response** message.
- Advantage of using the synchronous service is that the client application knows the status of the Web service operation in a very short time.
- When a Web service client sends a synchronous request to server, the response will be sent back to the client as soon as the process completes.



Asynchronous Web Services

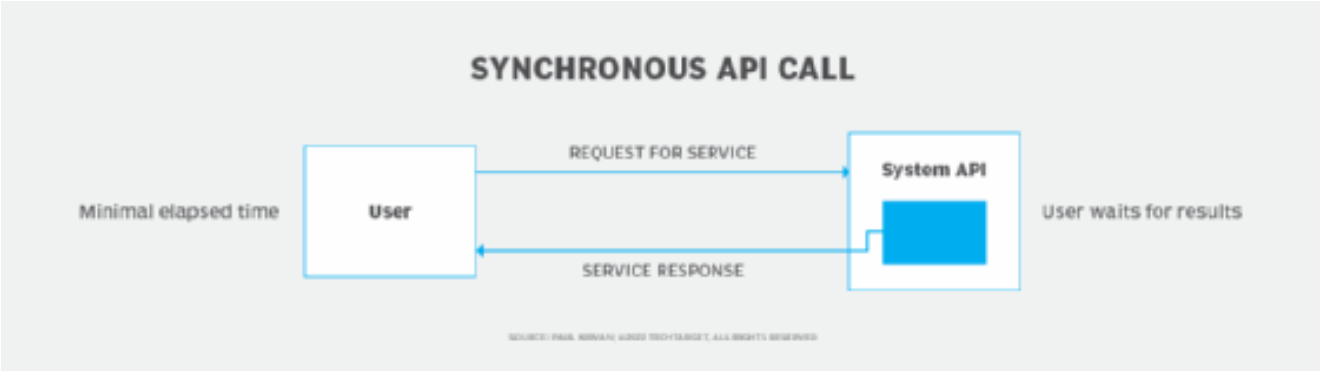
- This service execution **may require a significant amount of time to process a request.**
- However, the client that invoked the Web service **can continue with other processing in the meantime** rather than wait for the response.
- Non-blocking architecture, so the execution of one task isn't dependent on another. Tasks can run simultaneously.
- This pattern does not expect acknowledgment from client as it is a **fire-and-forget** message exchange pattern for callback.

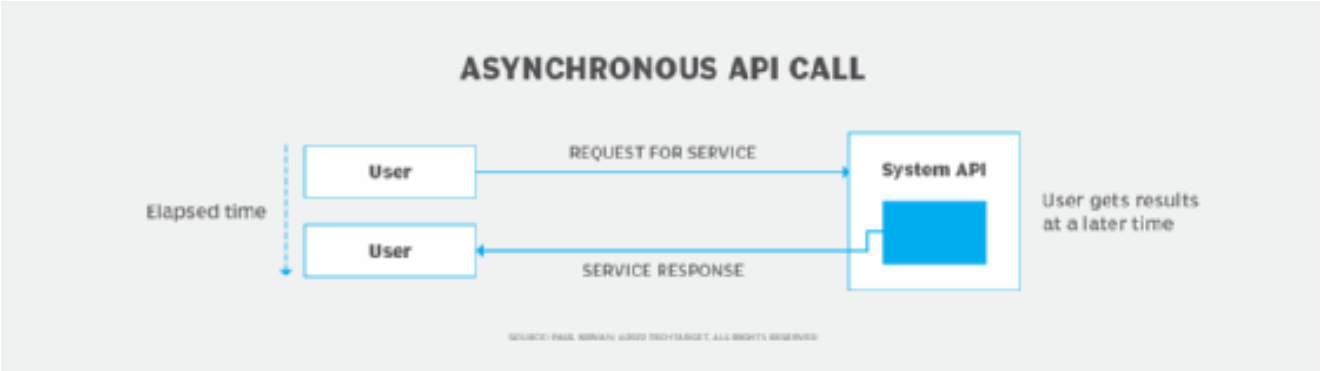


How do synchronous and asynchronous APIs work?

Telephone call is an example of **synchronous** communications, the parties are connected in real time. The two parties communicate back and forth in the same time frame.

Email is an example of **asynchronous** communications. The sender sends their message, but the recipient might not respond for some time.





CRESCENDO®
INTERNATIONAL
COLLEGE
IPTS No. DK006(J)

What is Celery?

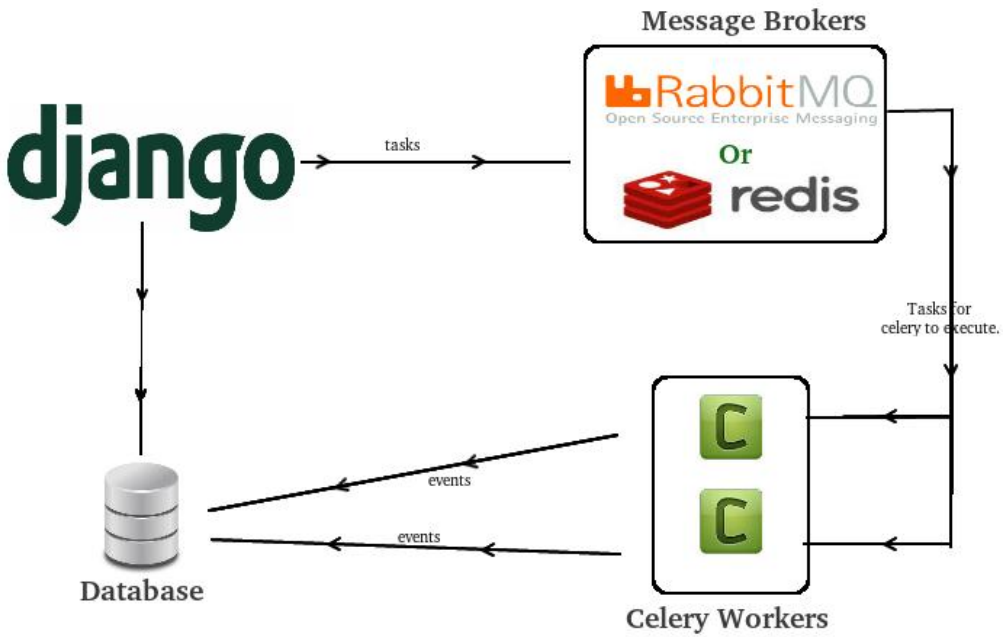
- Celery is a **task queue** implementation for Python web applications **used to asynchronously** execute work outside the HTTP request-response cycle.
- **Task queues** are used as a mechanism to distribute work across threads or machines.
- Celery **communicates via messages**, usually using a broker (transport) **to mediate between clients and workers.**
- To initiate a task the client adds a message to the queue, the broker then delivers that message to a worker.
- Celery is written in Python, but the protocol can be implemented in any language. In addition to Python there's node-celery and node-celery-ts for Node.js, and a PHP client.

https://docs.celeryq.dev/en/stable/getting-started/introduction.html#get-started

CRESCENDO®
INTERNATIONAL
COLLEGE
IPTS No. DK006(J)

What is uses Celery?

- **Reduce load** from the app and do that task in the background independently.
- **Scheduling tasks** at a specific time.
- Celery can help run tasks on the worker process instead of the web process.
- The web process can return the HTTP **response back immediately** (even if the task in the worker process is still running) to our user, and the **request cycle would not be blocked and the user experience would be better.**



CRESCENDO®
INTERNATIONAL
COLLEGE
IPTS No. DK006(J)

First Steps with Celery

- Choosing and installing a **message transport (broker)**.
  - RabbitMQ
  - Redis
- Installing Celery and creating your first task.
- Starting the worker and calling tasks.
- Keeping track of tasks as they transition through different states, and inspecting return values.

https://docs.celeryq.dev/en/stable/getting-started/first-steps-with-celery.html#application

CRESCENDO®
INTERNATIONAL
COLLEGE
IPTS No. DK006(J)

Download and Install

6.107 Installing Celery

```
pip install -U "celery[redis]==5.0.5
pip install redis==3.5.3
pip install Pillow==7.2.0
sudo apt-get install redis
```

pip install -U celery
pip install -U pillow
pip install redis

CRESCENDO®
INTERNATIONAL
COLLEGE
IPTS No. DK006(J)

Run redis (redis-server)

```
(test_env) PS C:\Users\HP\Desktop\Adv Web\libraryweb> redis-server
[18076] 12 Jan 15:58:26.529 # Warning: no config file specified, using the default config. In order to specify a config file use C:\
Program Files\Redis-x64-3.2.100\redis-server.exe /path/to/redis.conf

                _._
           _.-``__ ''-._
      _.-``    `.  `_.  ''-._           Redis 3.2.100 (00000000/0) 64 bit
  .-`` .-```.  ```\/    _.,_ ''-._
 (    '      ,       .-`  | `,    )     Running in standalone mode
 |`-._`-...-` __...-.``-._|'` _.-'|     Port: 6379
 |    `-._   `._    /     _.-'    |     PID: 18076
  `-._    `-._  `-./  _.-'    _.-'
 |`-._`-._    `-.__.-'    _.-'_.-'|
 |    `-._`-._        _.-'_.-'    |           http://redis.io
  `-._    `-._`-.__.-'_.-'    _.-'
 |`-._`-._    `-.__.-'    _.-'_.-'|
 |    `-._`-._        _.-'_.-'    |
  `-._    `-._`-.__.-'_.-'    _.-'
      `-._    `-.__.-'    _.-'
          `-._        _.-'
              `-.__.-'

[18076] 12 Jan 15:58:26.545 # Server started, Redis version 3.2.100
[18076] 12 Jan 15:58:26.545 * The server is now ready to accept connections on port 6379
```

Cont..

ADD IMAGES TO THE PROJECT

CRESCENDO®
INTERNATIONAL
COLLEGE
IPTS No. DK006(J)

Add Image Cover

- Go to the previous lab activity.
- Add the image cover.

```
class Book(models.Model):
    isbn = models.CharField(max_length=20, null=False, blank=False, db_index=True)
    title = models.CharField(max_length=200, null=False, blank=False)
    publication_year = models.IntegerField(null=True, blank=True)
    price = models.DecimalField(max_digits=6, decimal_places=2)
    author = models.ForeignKey(Author, on_delete=models.CASCADE)
    publisher = models.ForeignKey(Publisher, on_delete=models.DO_NOTHING)
    category = models.ForeignKey(Category, on_delete=models.DO_NOTHING)

    # Add new attribute for image
    cover_image = models.ImageField(upload_to ="image_books/", null=True, blank=True)

    def __str__(self):
        return self.title
```

CRESCENDO®
INTERNATIONAL
COLLEGE
IPTS No. DK006(J)

Update views.py

- Update the create_book for image.
- request.FILES must add to access images

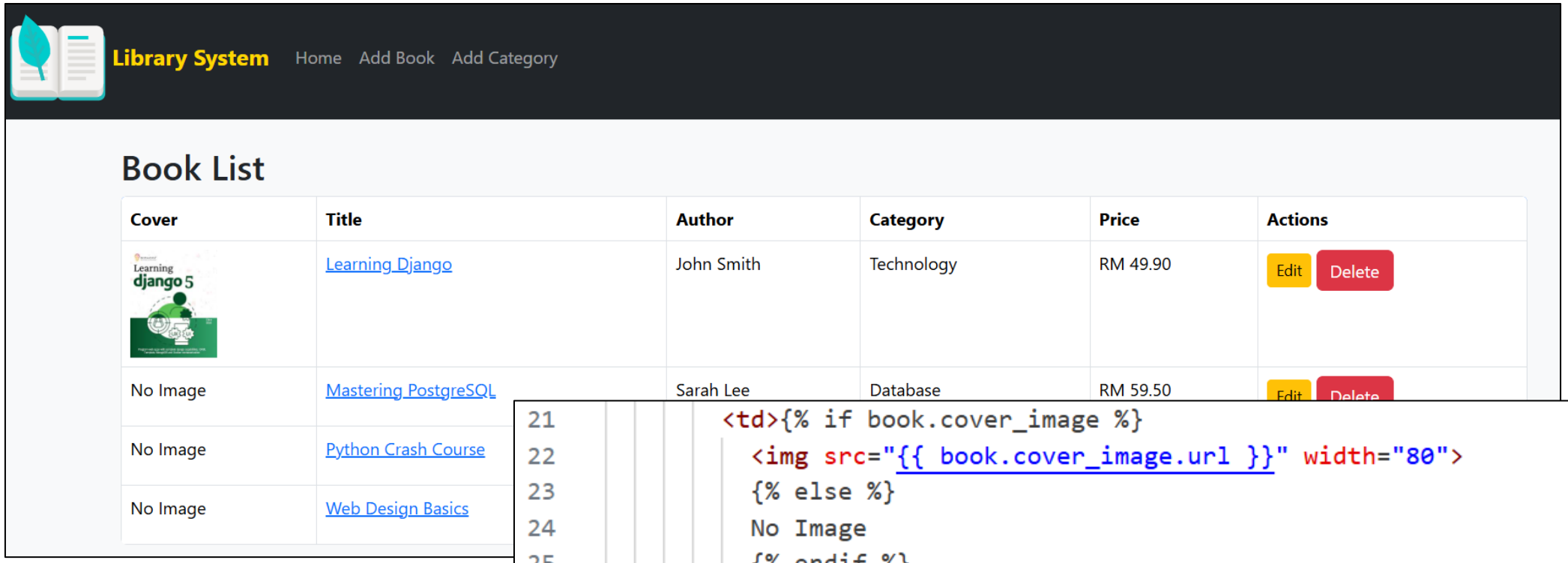
```
def create_book(request):
    if request.method == 'POST':
        form = BookForm(request.POST, request.FILES)
        if form.is_valid():
            form.save()
            return redirect('create_book')
    else:
        form = BookForm()

    books = Book.objects.all().order_by('-id')
    return render(request, 'books/create_book.html', {
        'form': form,
        'image_books': books
    })

```

CRESCENDO®
INTERNATIONAL
COLLEGE
IPTS No. DK006(J)

Modify form for add the cover book




```
21        <td>{% if book.cover_image %}
22          <img src="{{ book.cover_image.url }}" width="80">
23          {% else %}
24          No Image
25          {% endif %}
26        </td>
```

CRESCENDO®
INTERNATIONAL
COLLEGE
IPTS No. DK006(J)

Update url.py (library)

- This required to display all the information.

```
from django.contrib import admin
from django.urls import path, include
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('', include('books.urls')),
    path('admin/', admin.site.urls),
]

if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

CRESCENDO®
INTERNATIONAL
COLLEGE
IPTS No. DK006(J)

Update settings.py

- Add the setting for access the media image.
- That media folder will be created.

```
# setting for media image
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'

```

CRESCENDO®
INTERNATIONAL
COLLEGE
IPTS No. DK006(J)

Why we need Celery, Redis and Pillow?

- **Pillow** is a Python library that allows Django to handle images. Without Pillow, Django cannot process the ImageField in your models.
- **Celery** is an asynchronous task queue. It allows Django to "delegate" work so the user doesn't have to wait for a page to load.
- **Redis** is acts as the **Broker** between Django and Celery. It holds the task in a queue the pass messages between Django and Celery.

If I not implement all the tools what happens?

Django project will still work, but you will hit very specific **errors** as soon as you try to handle images or complex tasks.

CRESCENDO®
INTERNATIONAL
COLLEGE
IPTS No. DK006(J)

Flow In Project Library

Uploading and resizing images can be slow. Celery runs it in the background so the website stays fast.

User uploads image of books → Django saves the image → Celery task triggered → Pillow will resizes image → Redis will handles task queue

CRESCENDO®
INTERNATIONAL
COLLEGE
IPTS No. DK006(J)

Create celery.py and init.py

- This file for setting the celery for libraryweb project

celery.py | __init__.py | settings.py | create_book.html | tasks.py

libraryweb > libraryweb > celery.py

```python
import os
from celery import Celery

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'libraryweb.settings')

app = Celery('libraryweb')
app.config_from_object('django.conf:settings', namespace='CELERY')
app.autodiscover_tasks()

```

celery.py | __init__.py | settings.py

libraryweb > libraryweb > __init__.py

```python
from .celery import app as celery_app
__all__ = ('celery_app',)
```

CRESCENDO®
INTERNATIONAL
COLLEGE
IPTS No. DK006(J)

Update setting.py

- Add redis server into setting.py

```
# setting for redis server
CELERY_BROKER_URL = 'redis://localhost:6379/0'
CELERY_ACCEPT_CONTENT = ['json']
CELERY_TASK_SERIALIZER = 'json'
```

CRESCENDO®
INTERNATIONAL
COLLEGE
IPTS No. DK006(J)

Lab Activity

- Create image for cover book by implementing PILLOW, Redis and celery.
- Follow the command exercise:
  - 6.202: Setting up an image store
  - No need to create folder, just implement in the same folder.
- After that, differentiate between normal implementation vs implementing PILLOW, Redis and celery.

CRESCENDO®
INTERNATIONAL
COLLEGE
IPTS No. DK006(J)

What is Channel?

- Channels is a project that takes Django and extends its abilities beyond HTTP to handle WebSockets, chat protocols, IoT protocols, and etc.
- Django Channels allows Django to handle:
  - **WebSockets** (real-time communication)
  - Background events
  - Live updates (chat, notifications, task status)
- https://channels.readthedocs.io/en/stable/introduction.html
- Install
  - pip install channels
  - pip install channels-redis

CRESCENDO®
INTERNATIONAL
COLLEGE
IPTS No. DK006(J)

Lab

- Create chat channel for previous project (Library)
- Follow the command exercise:
  - 6.401 & 6.402 Installing and configuring a channels app
  - No need to create new project, just using the same project and app (Library)

```
INSTALLED_APPS = [
    'channels',
    'books',
    'bootstrap4',
    'rest_framework',
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
]
```